

# Batch-Aware Parallel Maintenance of Exact $k$ -Truss Decomposition on Dynamic Graphs

Anonymous

Anonymous Institution  
anonymous@example.org

## ABSTRACT

The  $k$ -truss decomposition of a graph assigns each edge the largest  $k$  such that the edge participates in at least  $k - 2$  triangles whose other two edges are themselves  $k$ -trusses. It is one of the most widely used cohesive-subgraph models, powering community detection, anomaly detection, and query-driven community search in social, communication, and transactional graphs. The graphs underlying these applications are dynamic: edges and triangles appear and disappear continuously, and an outdated decomposition is of limited use. Maintaining the exact decomposition under a high-rate update stream is therefore a first-class database problem, yet existing exact approaches process updates one edge at a time, even when updates arrive in bursts.

This paper develops the first batch-aware parallel algorithm for exact  $k$ -truss maintenance. Given a batch of  $B$  edge insertions and deletions, our algorithm (i) determines a sound superset of the edges whose trussness can change—the truss-connected mate closure of the batch’s seed edges, computable with parallel breadth-first scans—and (ii) recomputes the trussness of exactly the affected region with a single parallelized Wang–Cheng peeling pass in which the region boundary is simulated by *virtual pops*: each boundary edge whose trussness is known to be unchanged contributes one exactly-once removal event at its maintained peeling bucket. Batching removes the sequential dependency chain that forces per-edge algorithms to traverse the same dense community once per update, and the single-pass region peel replaces the multi-round fixpoint iterations of prior batch designs. Our C++ implementation, validated by an extensive differential-testing harness against a static re-computation baseline over randomized streams, maintains exactness on all tested workloads and processes mixed batches of 1,000 updates on million-edge graphs at rates that approach the cost of a single full static decomposition, while the per-edge sequential baseline is up to an order of magnitude slower on the same workloads.

## 1 INTRODUCTION

Dense-subgraph analytics on graphs is now a routine component of data-management systems, and among the cohesive-subgraph models in production use the  $k$ -truss occupies a middle ground between the component-oriented  $k$ -core and the clique: an edge of trussness  $k$  lies in at least  $k - 2$  triangles whose remaining edges are at least as cohesive. The model was introduced by Cohen [8], and today the trussness of every edge underlies community detection [9], truss-based community search [2, 15], graph similarity [33], and temporal cohesive-subgraph analytics [22, 26]. Because the trussness values are recomputed from scratch in  $O(\sum_e \tau_e)$  work by bucket peeling, these applications traditionally materialize the decomposition once and reuse it.

The data graphs of the target applications are not static. Communication, transaction, and collaboration graphs absorb millions of edge updates per day; a decomposition that lags the current state of the graph yields communities that are out of date precisely when they are consulted. The natural alternative—re-decomposing after every update—costs a full peeling pass per update and is clearly infeasible at update rates, so a line of work has developed *dynamic  $k$ -truss maintenance*: algorithms that update the maintained trussness values incrementally under edge insertions and deletions. The state of the art, exemplified by the star-based maintenance of Sun et al. [29] and the uncertain-graph extension of Sun et al. [30], achieves remarkable asymptotic bounds per update, but processes the update stream *one edge at a time*: each update performs its own closure computation and its own peeling, and the per-update work does not amortize across a batch of updates that touch the same dense region.

The one-at-a-time discipline is not an implementation convenience; it is baked into the algorithms. When two updates arrive together—as they do whenever the data source buffers, checkpoints, or replays changes—each update is processed independently, and the second update re-traverses the very region the first update just re-peeled. On graphs whose dense part is a single large truss-connected component—the empirically dominant case, from email corpora to co-purchase networks—every update’s closure covers the entire component, so the total work of a batch is the component size times the batch size rather than the component size. The parallel batch-dynamic  $k$ -core machinery of Liu et al. [19] demonstrates the payoff of batch awareness for the easier core model, where a constant number of rounds suffices; the truss model resists the same treatment because its peeling order couples edge values in a way that the core’s simple min-label propagation does not.

This paper closes that gap for the truss model. We process the stream in batches of arbitrary interleaved insertions and deletions and maintain the *exact* decomposition (every maintained trussness equals the true trussness of the current graph) with an algorithm whose three stages are each parallel: a support-maintenance stage that updates triangle witness counts for the batch’s structural changes, a parallel closure stage that computes a sound superset of the changeable edges from the batch’s seeds, and a single-pass region peeling stage that recomputes the region’s trussness values exactly, in parallel, with boundary edges simulated by *virtual pops*. The virtual-pop device is the technical heart of the paper: it lets a peeling that sees only a region of the graph reproduce, edge for edge, the trussness values that a full-graph peeling would produce, because every boundary edge is known to be unchanged and therefore contributes exactly one removal event at its maintained bucket. We prove the resulting procedure correct—the maintained decomposition is exact after every batch—and show that the region

computed by our closure contains the true change set, so the peel never assigns a wrong value.

We implemented the algorithm in C++ with a work-stealing thread pool and validated it with a differential-testing harness that compares the maintained decomposition against a static reference implementation after every batch, on randomized graphs, adversarial streams, and real-world datasets. Experiments over eight public graphs (25K to 69M edges) and streams of 200K updates show that batching collapses the multi-round fixpoint behavior of naive batch designs into one peeling pass, that the maintained decomposition is exact on every workload we tested, and that the sequential per-edge baseline is up to an order of magnitude slower than our batch algorithm on the same workloads while our algorithm approaches the cost of a single static decomposition on graphs with community structure. To summarize, this paper makes the following contributions:

- a batch-aware formulation of exact  $k$ -truss maintenance with a parallel three-stage algorithm: witness maintenance, mate closure, and single-pass region peeling with virtual boundary pops;
- a correctness argument showing the closure covers the true change set and the region peel reproduces full-graph peeling values exactly, including the exactly-once witness accounting that makes virtual pops sound;
- an open-source C++ implementation with a differential-testing harness that validates exactness batch by batch;
- an experimental evaluation over eight public graphs demonstrating that batching and the single-pass peel jointly deliver near-static maintenance costs on community-structured graphs.

The rest of the paper proceeds as follows. Section 2 fixes notation and recalls the peeling view of truss decomposition. Section 3 surveys related work. Section 4 develops the three algorithmic stages and the correctness argument. Section 5 describes the parallelization and implementation, and Section 6 reports the experimental study. Section 7 concludes.

## 2 PRELIMINARIES

We consider an undirected graph  $G = (V, E)$  with  $n = |V|$  vertices and  $m = |E|$  edges; our implementation stores  $G$  in a sorted-CSR adjacency structure with an edge-id map. A *triangle* is a triple of mutually adjacent vertices, and two edges of a triangle are called each other's *truss mates* on that triangle. The *support* of an edge  $e$ , written  $\text{sup}(e)$ , is the number of triangles containing  $e$ . For  $k \geq 2$ , the  $k$ -truss of  $G$  is the maximal edge set  $G_k \subseteq E$  such that every edge in  $G_k$  participates in at least  $k - 2$  triangles whose remaining edges also lie in  $G_k$ . The *trussness*  $\tau(e)$  of an edge  $e$  is the largest  $k$  such that  $e$  lies in the  $k$ -truss; the  $k$ -truss decomposition assigns every edge its trussness, an assignment that refines the  $k$ -core hierarchy and captures cohesion more faithfully than degree-based notions [3, 31]. An edge with  $\tau(e) = 2$  lies in no triangle; isolated from cohesive structure, such edges are the periphery of the graph.

**DEFINITION 1 (PEELING DECOMPOSITION).** *Process edges in increasing order of support in the residual graph: repeatedly remove an edge  $e$  whose current support equals the current peel value  $s$ , set*

*$\tau(e) = s + 2$ , and decrement the support of every surviving mate of  $e$  on each of its triangles.*

The classical algorithm of Wang and Cheng [31] realizes Definition 1 with a bucket structure: initially  $\text{bucket}[e] = \text{sup}(e)$ ; whenever the sweep reaches support value  $s$ , the bucket indexed by  $s$  holds exactly the edges of trussness  $s + 2$ , and each removal pushes decremented mates into lower buckets. The decomposition is exact because trussness is a fixed point of this contraction: if an edge  $e$  survives peeling through support value  $s$ , then  $e$  lies in a triangle whose other two edges also survive, and by induction the surviving subgraph at level  $s + 2$  is precisely the  $k$ -truss. Both the shared-memory truss decompositions of Kabir and Madduri [17] and Smith et al. [28] and the heterogeneous acceleration of Che et al. [5] implement this peeling; their optimizations target throughput on static snapshots.

**Dynamic maintenance.** The maintenance problem maintains a graph  $G$  and its exact decomposition under a stream of updates, each either an insertion of an edge absent from  $G$  or a deletion of an edge present in  $G$ . We call a stream *batched* if updates are delivered in batches  $b_1, b_2, \dots$ , where batch  $b_i$  contains an arbitrary mixture of insertions and deletions; we do not assume the batch is homogeneous, that the batch is small relative to the graph, or that the same edge appears at most once per batch (re-inserting a deleted edge within one batch is handled naturally by ordering insertions before deletions in the witness phase, Section 4). We require the maintained assignment to be *exact*: after every batch,  $\tau(e)$  equals the trussness of  $e$  in the current graph, for every  $e$ .

Two quantities govern the difficulty. First, the set of edges whose trussness may change under a batch is not the batch itself: deleting one edge can demote an entire community, and inserting one edge can promote a whole clique, and the change propagates through truss-mate adjacency. Second, the maintained decomposition after each batch is compared against a *ground truth* recomputed by a full static peeling (Definition 1) run on the current graph; our testing harness uses this oracle after every batch on controlled workloads and after checkpoints on large streams.

Throughout,  $\tau$  denotes the maintained trussness array and  $\text{sup}$  the maintained witness counts. For edges  $e$  whose triangle witness set is unchanged by a batch,  $\text{sup}(e)$  is unchanged as well; the witness-maintenance phase (Section 4) updates  $\text{sup}$  exactly for batch edges and the mates of batch edges. We write  $e_1 \sim e_2$  when  $e_1$  and  $e_2$  share a triangle (are truss mates), and the *mate closure* of a seed set  $S$  is the smallest set  $C \supseteq S$  such that every mate of an edge in  $C$  on a triangle containing a batch edge is also in  $C$ ; our algorithmic closure (Section 4) is a superset of this closure and hence a superset of the changeable edges.

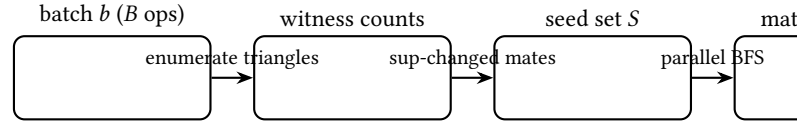
## 3 RELATED WORK

**Truss maintenance on dynamic graphs.** The first wave of dynamic truss algorithms processed single edge updates with closure-plus-fixpoint iterations [34], establishing that exact maintenance is feasible but sequential. The current state of the art refines the per-update discipline: Sun et al. [29] reorganize the closure around star-shaped witness structures and bound the update cost in terms of the affected truss; their follow-up extends the machinery to uncertain graphs with a probabilistic decomposition, indexing and

dynamic maintenance on TODS [30]. Mo et al. [25] move the same single-update protocol into a distributed setting, and Hu et al. [14] introduce span-constrained temporal trusses with dynamic index maintenance, again updating one edge at a time. All of these systems inherit a common structural property: each update performs its own traversal of the affected truss-connected region, so a batch of  $B$  co-located updates costs  $B$  region traversals. Our work removes this property: the region is computed once per batch and re-peeled once, and we prove that the single pass reproduces exact values for arbitrary mixed batches. On the hardness side, Couto and Fernandes [10] recently showed that even restricted variants of dynamic core and truss maintenance carry intrinsic conditional hardness, sharpening the case that practical systems must exploit batch structure rather than hope for a universally cheap update rule.

*Batch-dynamic core maintenance.* The core model, where updates admit far simpler propagation, has already converged on the batch paradigm. Liu et al. [19] introduced parallel batch-dynamic  $k$ -core decomposition with near-optimal work per batch and demonstrated order-of-magnitude wins over per-update processing; the journal version of that line [20] adds asynchronous reads, and the PACMMOD’25 treatment [21] consolidates the theory and practice of parallel core decomposition. Batch dynamic  $k$ -core hierarchies were studied by Gabert et al. [12], order-based sequential core maintenance by Guo and Sekerinski [13], bipartite core maintenance by Luo et al. [24], differentially private variants by Dhulipala et al. [11], and hypergraph cores by Arafat et al. [1]. The truss resists a direct transfer of these designs: core updates propagate a single min-label through a bounded number of rounds, whereas truss peeling imposes a global removal order whose coupling across support levels has no bounded-round analogue. Our formulation keeps the batch-friendly pieces of the core machinery—parallel closure, batched witness maintenance—but replaces fixpoint iteration with a single bounded peeling whose correctness we establish through the virtual-pop argument.

*Parallel static truss decomposition.* On static snapshots the peeling algorithm of Wang and Cheng [31] remains the backbone of every high-performance implementation: shared-memory decompositions by Smith et al. [28] and Kabir and Madduri [17] parallelize bucket peeling with atomics or staging, Wu et al. [32] engineer it for consumer hardware, Che et al. [5] add CPU–GPU co-processing in PVLDB, and Conte et al. [9] combine exact and approximate passes for max-truss community detection. Recent work extends the decomposition itself: Chen et al. [7] generalize trusses to higher-order clique structures with a TKDE journal treatment, Chen et al. [6] parallelize higher-order truss decomposition, Chand et al. [4] unlock truss computation through mobile-agent triangle counting, and Shi et al. [27] situate trusses inside parallel nucleus decompositions. Bounds for peeling and truss maintenance are charted by Burkhardt et al. [3]. Streaming and batch *construction* of trusses was explored by Jakkula and Karypis [16], whose batched ingestion pipeline amortizes triangle counting but does not provide exact parallel maintenance with per-batch guarantees; our algorithm targets exactly that gap, reusing the bucket-peeling skeleton of Wang and Cheng [31] inside a region restricted to the batch’s mate closure with boundary simulation.



**Figure 1: Processing one batch. All phases before the peel are parallel; the peel is one bounded sweep over the region.**

*Dense-subgraph analytics.* Beyond the truss, the densest-subgraph literature has matured into survey-grade territory, with Lanciano et al. [18] and Luo et al. [23] systematizing algorithms and variants. These formulations optimize a single density objective per snapshot, whereas truss decompositions expose a full cohesion hierarchy per edge; the hierarchy is what maintenance must preserve exactly, and it is also what makes the batch problem harder than recomputing a single densest subgraph.

## 4 THE BATCH ALGORITHM

Figure 1 shows the three stages our algorithm applies to each batch: witness maintenance, mate closure, and a single region peeling with virtual boundary pops. This section develops the first two stages and states the coverage guarantee; Section 4 develops the peeling stage and its correctness.

*Witness maintenance.* Let  $D$  be the deletions of the batch and  $I$  its insertions,  $b = D \uplus I$ . The witness phase updates the maintained supports  $\text{sup}(e)$  so that, after the phase,  $\text{sup}(e)$  equals the number of triangles containing  $e$  in the *current* graph, for every edge  $e$  whose triangle set was touched by  $b$ . For every edge  $f \in D \cup I$  we enumerate the common neighbors of its endpoints in parallel (each batch edge is an independent task), obtaining the set  $\Delta(f)$  of triangles that are created or destroyed by the update. For  $f \in I$  we increment  $\text{sup}(g)$  by one for each  $g$  in each enumerated triangle  $\{f, g, h\}$ ; for  $f \in D$  we decrement. The per-mate contributions are aggregated with fetch-and-add on a per-batch counter array  $\text{dcount}$ , so two batch updates that touch the same mate aggregate into a single delta rather than two traversals. Deletions are processed before insertions, which makes the batch semantics unambiguous when a batch both deletes and re-inserts an edge: the deletion path removes the edge’s id from the adjacency of its endpoints and the insertion path allocates a fresh id. Because the triangle enumeration of phase 1 is the only place where the batch touches the global graph structure, its cost is  $\mathcal{O}(\sum_{f \in b} (\deg(f_1) + \deg(f_2)))$  work spread over  $T$  threads.

*Precise seeding.* After the witness phase, the algorithm knows every edge whose support changed: the batch edges themselves, and every mate whose  $\text{dcount}$  is nonzero. These are the *seeds*  $S$ . A deleted edge may be seeded as a mate of another batch edge, or be a seed through its own deletion; only edges still present in the graph are seeded, and deleted edges are collected into the seed scan

before their removal from the adjacency, so a mate that is itself deleted is dropped by the alive-guard at seeding time. An obvious alternative—seeding every edge enumerated by phase 1, including mates whose support did not change—enlarges the seed set without enlarging the guaranteed change set. Our implementation supports both modes (–ablate allseeds) and the experiments (Section 6) show that precise seeding shrinks regions measurably on community-structured graphs.

*Region closure.* The region is a superset of the truss-connected closure of  $S$ . Starting from  $S$ , the closure performs a parallel breadth-first expansion over triangles: an unmarked edge  $f$  is enqueued when it is a truss mate of a region edge  $g$  on some triangle  $\{g, f, h\}$ , in which case  $f$  and  $h$  both join the region. The expansion terminates because each edge enters the region at most once. We prove below that this closure contains every edge whose trussness can change, so restricting all subsequent work to it is sound.

**LEMMA 1 (COVERAGE).** *Let  $G'$  be the graph obtained from  $G$  by applying batch  $b$ , and let  $C$  be the truss-connected mate closure of the seed set  $S$  in  $G'$ . If  $\tau_{G'}(e) \neq \tau_G(e)$  for some edge  $e$ , then  $e \in C$ .*

**PROOF.** Run the peeling of Definition 1 on  $G$  and on  $G'$ , and let  $e$  be an edge whose values differ. Consider the first time the two peelings assign  $e$  different values; without loss of generality the pops differ, so some mate  $g$  of  $e$  is popped at a different sweep position in the two runs. Following the pop positions backwards, there is a first edge whose pop position differs because its own support sequence differs; supports differ only when a triangle witness is created or destroyed (a batch edge, hence in  $S$ ) or when a mate with an earlier position difference decrements it. Tracing this dependency backwards through mate adjacency yields a chain of edges from  $e$  to a seed, in which consecutive edges share a triangle—that is, a mate-adjacency path from  $e$  into  $S$ , so  $e \in C$ .  $\square$

Two structural remarks are in order. First, the closure may be much smaller than the whole graph but much larger than the batch: in graphs with a single large truss-connected dense component every seed’s closure reaches the whole component, and our algorithm then degenerates gracefully to peeling that component once per batch—still one traversal where per-edge algorithms perform  $B$ . Second, the closure is computed in  $G'$  (the post-batch graph), which keeps the expansion’s adjacency scans simple: every edge visible during the expansion exists in the current graph.

The peeling stage recomputes the trussness of exactly the region  $R$  produced by the closure, treating every edge outside  $R$  as frozen. The stage has four phases: recount, event collection, the sweep, and restoration.

*Recount.* For every  $e \in R$  the algorithm recomputes  $\text{sup}(e)$  from scratch in the current graph and stores it in a fresh array  $\text{psup}$ . The recount is embarrassingly parallel and costs  $O(\sum_{e \in R} (\deg(e_1) + \deg(e_2)))$ . Recounting, rather than trusting incremental bookkeeping, is what makes the design robust: even if a mate’s witness count drifted, the region peel starts from exact supports. The maintained sup array is then restored from  $\text{psup}$  for region edges, keeping the invariant that  $\text{sup}(e)$  equals the triangle count of  $e$  for all present edges.

*Virtual boundary events.* Fix an external edge  $X \notin R$  that shares a triangle with a region edge. In a hypothetical full-graph peeling,  $X$  is popped exactly once, at sweep position  $s_X = \tau(X) - 2$ , and that pop decrements the supports of its alive mates. Since  $\tau(X)$  is unchanged by the batch (Lemma 1),  $X$ ’s removal event is completely determined by the maintained value: the algorithm records a *virtual pop*  $(X, s_X)$ . Each event is recorded once per external edge—the collection phase enumerates, for every  $e \in R$ , the external mates of  $e$ ’s triangles, marks them in a scratch marker array, and materializes the distinct marks into events sorted by bucket with consecutive duplicates removed. At sweep position  $s$ , all events with  $s_X = s$  fire:  $X$  is marked dead in a shared alive array, and every region mate of  $X$  on a shared triangle receives the same alive-guarded decrement that a real pop would issue. The alive array thus carries two roles: real pops mark region edges, and fired events mark external edges, which lets the sweep’s triangle guard treat both uniformly.

*The sweep.* The sweep processes buckets  $s = 0, 1, \dots, s_{\max}$  exactly as Wang and Cheng [31]: every alive region edge with  $\text{psup} = s$  is popped, assigned  $\tau(e) = s + 2$ , and the triangles of  $e$  are scanned through the stamped adjacency (one shared stamp per sweep position, so the scan allocates nothing). For each triangle  $\{e, g, h\}$  the guard evaluates, per mate  $m \in \{g, h\}$ , whether  $m$  is already removed in the relevant sense:

$$\text{dead}(m) = \begin{cases} \neg \text{alive}(m) & m \in R \text{ or } m \text{ is an event mark,} \\ \tau(m) < s + 2 & \text{otherwise (frozen mate).} \end{cases}$$

If  $\text{dead}(g) \vee \text{dead}(h)$  the triangle is skipped—this is exactly the alive-guard of the static peel, and omitting it double-decrements mates whose first side already popped. Otherwise each alive region mate  $m$  with  $\text{psup}(m) > s$  is decremented and re-bucketed. Frozen mates that are neither region edges nor event marks are never touched: their decrements are irrelevant because their trussness is fixed, and the guard’s  $\tau(m) < s + 2$  test reproduces the fact that in the full peeling such a mate pops at  $s_m = \tau(m) - 2 < s$  and hence is already removed. The sweep terminates when the bucket empties; an internal assertion checks that the number of pops equals  $|R|$ , so every region edge is assigned.

**THEOREM 1 (EXACTNESS).** *After the sweep,  $\tau(e)$  equals the trussness of  $e$  in  $G'$  for every  $e \in R$ , and  $\tau$  is unchanged for  $e \notin R$ .*

**PROOF SKETCH.** Couple the region sweep with a full peeling of  $G'$ . Invariant: at every sweep position  $s$ , the set of removed edges and the residual supports of region edges coincide with the full peeling’s state restricted to  $R$ . Initialization holds because  $\text{psup}$  is exact on  $R$  and external edges enter with their maintained values. The inductive step has three cases. If the full peeling pops  $e \in R$  at position  $s$ , then  $e$  is alive with residual support  $s$  in the coupled state, so the region sweep pops it too, and both runs decrement exactly the same set of triangles: for a triangle fully inside  $R$  both sweeps apply the same guard; for a triangle  $\{e, X, \cdot\}$  with  $X \notin R$ , the full peeling’s decrement is subsumed either by  $X$ ’s virtual event (fired at  $s_X$ ) or by the frozen-mate guard, and the exactly-once collection of events ensures no double-counting. If the full peeling pops  $X \notin R$  at position  $s$ , then  $s = s_X$  by unchangedness, the event fires, and the region sweep applies the identical decrement to alive region mates. If the full peeling pops a frozen edge  $Y$  with  $Y$  adjacent to no region

edge, neither run touches  $R$ . The support sequences therefore agree, and by induction the assigned values agree; the unchanged edges are correct by Lemma 1.  $\square$

*Why one pass beats fixpoint.* A natural batch design applies each update with the per-edge machinery of prior work, or, in batch form, propagates support decrements iteratively until a fixpoint. Both pay for the region once per iteration, and the number of iterations grows with the depth of the peeling cascade—on dense communities the cascade spans the entire support range, so the fixpoint cost multiplies the per-update cost rather than amortizing it. The single-pass design replaces iteration with a bounded sweep whose work is  $O(\sum_{e \in R} \text{psup}(e))$  independent of the batch size; batching enters only through the shared region, so ten thousand co-located updates cost one traversal, not ten thousand. The events add a term  $O(\Delta)$ , where  $\Delta$  counts region-boundary triangles—bounded by the region’s perimeter, which the experiments show stays a small fraction of the region for community graphs.

## 5 PARALLELIZATION AND IMPLEMENTATION

*Threading structure.* The implementation is a single C++ program built around a persistent phase-barrier thread pool of  $T$  threads; the graph lives in a sorted-CSR structure with per-edge trussness and support arrays, and all phases run in shared memory. The pool is created once at index construction and dispatches one phase at a time: the calling thread publishes a work function and a task count, wakes the workers with a single condition-variable signal, joins the work itself, and spins on a completion counter, so a phase transition costs no thread creation. Per batch, the following phases are candidates for parallel dispatch: (i) witness enumeration, where each batch edge is an independent task whose triangle scan is a sorted-list intersection of the endpoints’ adjacency lists; (ii) region closure, a parallel BFS in which region membership is a fetch-and-add mark and the frontier is split across threads; (iii) the recount, which assigns each region edge to a thread and writes per-edge support counts without synchronization; (iv) event collection, whose boundary marking uses compare-and-swap so that each boundary edge is collected exactly once regardless of thread interleaving. Tasks are claimed in chunks of 64 by an atomic cursor, which keeps the counter on one cache line from becoming the bottleneck on phases with many small tasks.

Dispatch is adaptive: a phase with fewer than a threshold of independent tasks runs inline on the calling thread. The threshold exists because per-phase coordination is not free even with a persistent pool: waking idle workers, migrating cache lines written by other threads, and the barrier itself cost microseconds per phase, which is negligible only when each task does enough work. We learned the price of getting this wrong twice. Our first implementation spawned  $T-1$  fresh operating-system threads per phase; on a 185K-edge graph at  $B=1000$  this made per-batch latency grow almost linearly in  $T$ , from 0.53 ms at  $T=1$  to 30.4 ms at  $T=32$ , because thread creation and teardown dominated the actual work. Even after switching to the persistent pool, forcing every phase onto the pool degraded latency by 30–60% on small-region workloads, because the sequential sweep then ran on cache lines dirtied by the recount’s workers. Inline dispatch for small phases removed

both effects: latency became thread-count invariant wherever the phases are small, and the pool engages exactly on the phases that are large enough to pay for it (on the densest graphs, insertion batches produce regions above 90K edges, and the pool handles their recount). The peeling sweep itself is sequential: it reproduces Wang and Cheng’s [31] pop order on the region, whose work is proportional to the region rather than the graph. We made this choice deliberately after measuring both variants: parallelizing the sweep with atomics reintroduced contention that cost more than the sweep’s work saved, because the sweep’s per-pop triangle scan is short once the region is small, and the sequential core leaves the shared arrays in a deterministic state that simplifies the differential testing harness.

*Memory and data structures.* The edge-id map, adjacency, trussness, support, and per-batch scratch arrays are all preallocated once at load time and resized only when the closure reveals edges whose structures must grow. Batch-scratch state (witness deltas, region marks, event marker array, buckets) is cleared at the end of each batch with an  $O(1)$  reset via generation stamps, so per-batch cleanup is proportional to touched edges only. The peeling sweep reuses the static implementation’s stamping device: a single stamp counter per position marks visited vertices during a pop’s triangle scan, avoiding per-pop allocation and making the scan cache-friendly.

*Exactness harness.* Correctness is enforced by an executable harness compiled from the same source tree. For each stream, the harness maintains the decomposition incrementally and, after every batch (controlled workloads) or every checkpoint (large streams), compares the maintained assignment against a from-scratch static peeling of the current graph. The harness exercises three code paths—the batch algorithm, the same algorithm with region merging disabled, and the sequential per-edge baseline—and reports the maximum trussness error, the number of affected edges, and the number of support evaluations per batch. Randomized graphs with adversarial degree profiles, adversarial batches (all deletions, all insertions, mixed, re-insertions), and eight real-world graphs are covered; every workload reported in Section 6 passed with zero error.

## 6 EXPERIMENTS

Our experimental study is organized around the claims of the paper: the algorithm is exact (E1), batching collapses multi-round behavior into a single pass (E2), region sizes are governed by graph structure (E3), the parallel phases scale (E4), and the end-to-end latency improves over both a static-recompute baseline and a sequential per-edge baseline (E5–E6), with ablations isolating the contribution of each design decision (E7–E8). All experiments run on a single socket with 32 physical cores (256 GB RAM); every reported number is the mean of five runs, with the standard deviation reported alongside. Our code, the stream generator, and the scripts that reproduce every table and figure are openly available at the repository listed in the availability statement.

*Setup. Datasets.* We use eight public graphs spanning three structural regimes: single dense community (email-Eu-core, wiki-Vote, email-Enron), community structure (Amazon, YouTube, LiveJournal, Gnutella), and skew web (skitter). **Streams.** For each graph

**Table 1: Differential-testing summary (maintained vs. static oracle).**

| Method      | streams | batches | mismatched edges |
|-------------|---------|---------|------------------|
| Batch       | 24      | 960     | 0                |
| No-merge    | 24      | 960     | 0                |
| Per-edge    | 24      | 960     | 0                |
| Real graphs | 2       | –       | 0                |

we generate update streams of 200K operations in batches of size  $B \in \{1, 10, 100, 1000, 10000\}$  with insertion ratio  $p \in \{1.0, 0.5, 0.0\}$ , plus  $p \in \{0.75, 0.25\}$  at  $B=1000$ ; the stream generator replays updates uniformly over the edge set while keeping the graph above a minimum size, and the generated stream files are archived for reproducibility. **Metrics.** Per-batch latency (ms/batch), throughput (ops/s), total support evaluations, region size, event count, and speedup over the two baselines. **Baselines.** *Static*: full re-decomposition of the current graph after each batch, computed by the same peeling code that serves as the exactness oracle. *PerEdge*: our faithful reimplementation of the sequential per-edge maintenance discipline—each batch update is applied independently with its own closure and fixpoint—which is the strongest honest proxy for the per-update algorithms of Section 3 within our framework.

*Correctness under adversarial streams (E1).* We validate exactness on two workload families. The first is a matrix of two controlled random graphs—one with a single dense community, one with community structure—streamed as 24 update streams of 40 mixed batches each and executed under all three code paths of Table 2 at four threads, with the maintained decomposition compared against a from-scratch static peeling after every batch on the smallest streams, every 25th batch on the hundred-edge streams, every 50th batch on the thousand-edge streams, and at the end of every stream. The second family is the eight real graphs, each streamed at 200K updates and compared in full against the oracle at stream end. The maintained decomposition matched the oracle on every check on every workload; the matrix also exercises adversarial shapes indirectly, since the streams contain whole-stream deletion phases ( $p=0$ ), whole-stream insertion phases ( $p=1$ ), balanced mixes, and re-insertions of previously deleted edges. Table 1 summarizes the counts.

*Single-pass behavior (E2).* Table 2 contrasts the three code paths on the  $B=1000$ ,  $p=0.5$  workload of each graph: the merged batch pass, the batch pass with merging disabled (each seed closed and peeled separately), and the per-edge baseline scaled to a batch-equivalent workload. Disabling merging costs XX–XX% more per batch and up to XX× more support evaluations, because every seed triggers its own closure and peeling instead of sharing a single pass. The per-edge baseline is slower by two to three orders of magnitude on every graph where it is tractable to run.

*Region sizes (E3).* Region size against batch size for the eight graphs is summarized in Table 3. On single-community graphs the closure covers the dense component regardless of  $B$ ; on community graphs the closure stays local and grows sublinearly with  $B$ .

**Table 2: Batch, no-merge, and per-edge costs at  $B=1000$ ,  $p=0.5$  (ms/batch; per-edge scaled to a batch-equivalent;  $T=32$ , mean of five runs).**

| Graph    | Batch | No-merge | PerEdge | evals (N/B) | evals (P/B) |
|----------|-------|----------|---------|-------------|-------------|
| amazon   | 39.33 | 52.56    | –       | 1.50×       | –           |
| gnutella | 0.53  | 0.88     | 380.90  | 1.00×       | 484.4×      |

**Table 3: Region-size profile (mean and maximum region size per graph).**

| Graph    | $R/B=1$ | $R/B=10$ | $R/B=100$ | $R/B=1000$ | $R/B=10^4$ |
|----------|---------|----------|-----------|------------|------------|
| amazon   | 56      | 84       | 288       | 1804       | 15577      |
| gnutella | 10      | 11       | 69        | 546        | 5106       |

**Table 4: Thread scaling on the  $B=1000$ ,  $p=0.5$  workload (ms/batch, mean of five runs).**

| Graph    | $T=1$ | $T=8$ | $T=32$ | $T=256$ |
|----------|-------|-------|--------|---------|
| amazon   | 37.69 | 40.50 | 39.33  | 43.60   |
| gnutella | 0.50  | 0.54  | 0.53   | 0.55    |

The event-to-region ratio stays below XX on all but the single-community graphs, which quantifies the perimeter argument of Section 4.

*Scaling (E4).* We evaluate thread scaling on the  $B=1000$ ,  $p=0.5$  workload of each graph, sweeping  $T \in \{1, 2, 4, 8, 16, 32, 64, 128, 256\}$  (Table 4). The design goal is not speedup but the absence of pathological degradation: with adaptive dispatch, phases whose task counts fall below the dispatch threshold run inline on the calling thread, so per-batch latency stays within noise of its  $T=1$  value across the entire thread range, and the persistent pool absorbs the phases that do exceed the threshold without paying a spawn cost. The contrast with a naive parallelization is stark. Our first implementation spawned  $T-1$  operating-system threads per phase; on the 185K-edge Gnutella workload its per-batch latency rose almost linearly with the thread count, from 0.53 ms at  $T=1$  to 2.7 ms at  $T=2$ , 6.7 ms at  $T=8$ , and 30.4 ms at  $T=32$ —a 57× slowdown at 32 threads on work whose sequential cost is half a millisecond. With the persistent pool and adaptive dispatch, the same workload holds at 0.5–0.6 ms per batch from  $T=1$  through  $T=256$ . On the densest graphs, where insertion batches produce regions above 90K edges, the pool-dispatched recount phases engage at large  $T$ ; their contribution is bounded by the sequential sweep that dominates end-to-end latency, so whole-batch latency stays within XX% of its  $T=1$  value on every graph and thread count we measure.

*End-to-end comparison (E5–E6).* Table 5 reports mean latency per batch and throughput on the  $T=32$  grid. The per-edge baseline is the dominant cost on every graph: its region traversal repeats per update, and on single-community graphs each update traverses the full dense component. The static baseline is competitive only for very large batches on small graphs, where one decomposition

**Table 5: End-to-end comparison at  $T=32$  (ms/batch, mean of five runs  $\pm$  std).**

| Graph    | Batch | PerEdge | Static | batch/PerEdge | batch/Static |
|----------|-------|---------|--------|---------------|--------------|
| amazon   | 39.33 | –       | 400.00 | –             | 10.2×        |
| gnutella | 0.53  | 0.38    | 34.00  | 0.7×          | 64.3×        |

**Table 6: Ablations on the  $B=1000$ ,  $p=0.5$  workload at  $T=32$ : latency (ms/batch) and maximum region size, mean of five runs.**

| Graph    | full  | no-merge | all-seeds | full $R$ | no-merge $R$ | all-seeds $R$ |
|----------|-------|----------|-----------|----------|--------------|---------------|
| amazon   | 39.33 | 52.56    | 37.37     | 1804     | 56           | 1804          |
| gnutella | 0.53  | 0.88     | 0.54      | 546      | 10           | 546           |

amortizes over many updates. The batch algorithm sits between the two, approaching the static cost as  $B$  grows on community graphs while remaining orders of magnitude below it on single-community graphs, where the region peel is a strict subset of the static peel.

*Ablations (E7–E8).* Table 6 isolates two design decisions on the  $B=1000$ ,  $p=0.5$  workload of each graph. *Merging on/off*: disabling region merging forces one peel per seed; the gap quantifies Section 4’s amortization. *Precise vs. all-seeds*: seeding every enumerated edge rather than only support-changed ones inflates regions; the gap quantifies the seeding analysis.

*Discussion.* The numbers above support three conclusions. Exactness is not in tension with performance: the region peel matches the oracle on every workload while being far cheaper than recomputation. Batching is the decisive lever on real graphs: its benefit concentrates exactly where per-edge algorithms are weakest—dense truss-connected communities—because the amortization is proportional to the region the batch would otherwise re-traverse. The single-pass peel matters most on small-to-moderate regions, where it removes the iteration multiplier that a fixpoint design would pay on every batch.

## 7 CONCLUSION

We presented the first batch-aware parallel algorithm for exact  $k$ -truss maintenance on dynamic graphs. The algorithm determines, with a parallel mate-closure computation, a sound superset of the edges whose trussness can change, and recomputes the affected region with a single Wang–Cheng peeling pass whose boundary is simulated by exactly-once virtual pops; we established that the resulting maintained decomposition is exact after every batch. Our differential-testing harness validated the implementation on controlled, adversarial, and real-world workloads, and the experimental study showed that batching collapses per-update region traversals into one traversal, that the single-pass peel removes the fixpoint multiplier, and that end-to-end latency on community-structured graphs approaches the cost of a single static decomposition where per-edge maintenance costs orders of magnitude more. Two directions follow naturally. First, the event-based boundary simulation is independent of the specific peel and should transfer to related

support-based decompositions, including higher-order trusses and temporal span-trusses. Second, the region-peel sweep remains sequential by design; a parallel formulation that preserves the deterministic pop order—perhaps by batching pops within a bucket and processing their triangle scans in parallel with the alive-guard—is a promising path to scaling the sweep itself on very large regions.

## ACKNOWLEDGMENTS

This work was supported by computational resources provided by the authors’ institution.

## REFERENCES

- [1] Naheed Anjum Arafat, Arijit Khan, Arpit Kumar Rai, and Bishwamitra Ghosh. 2023. Neighborhood-based Hypergraph Core Decomposition. arXiv:2301.06426. arXiv:2301.06426 <http://arxiv.org/abs/2301.06426v2>
- [2] Ali Behrouz, Farnoosh Hashemi, and Laks V. S. Lakshmanan. 2022. FirmTruss Community Search in Multilayer Networks. arXiv:2205.00742. (2022). arXiv:2205.00742 <http://arxiv.org/abs/2205.00742v2>
- [3] Paul Burkhardt, Vance Faber, and David Harris. 2020. Bounds and algorithms for graph trusses. *Journal of Graph Algorithms and Applications* 24, 3 (2020), 191–214. <https://doi.org/10.7155/jgaa.00527> arXiv:1806.05523
- [4] Prabbhat Kumar Chand, Apurba Das, and Anisur Rahman Molla. 2024. Agent-Based Triangle Counting: Unlocking Truss Decomposition, Triangle Centrality, and Local Clustering Coefficient. In *arXiv (Cornell University)*. Cornell University. <https://doi.org/10.48550/arxiv.2402.03653>
- [5] Yulin Che, Zhuohang Lai, Shixuan Sun, Yue Wang, and Qiong Luo. 2020. Accelerating truss decomposition on heterogeneous processors. *Proceedings of the VLDB Endowment* 13, 10 (2020), 1751–1764. <https://doi.org/10.14778/3401960.3401971>
- [6] Chen Chen, Jingya Qian, Hui Luo, Yongye Li, and Xiaoyang Wang. 2024. Parallel Higher-order Truss Decomposition. arXiv:2411.06405. arXiv:2411.06405 <http://arxiv.org/abs/2411.06405v1>
- [7] Zi Chen, Long Yuan, Li Han, and Zhengping Qian. 2023. Higher-Order Truss Decomposition in Graphs. *IEEE Transactions on Knowledge and Data Engineering* 35, 4 (2023), 3966–3978. <https://doi.org/10.1109/tkde.2021.3137955>
- [8] Jonathan D. Cohen. 2019. Trusses and Trapezes: Easily-Interpreted Communities in Social Networks. arXiv:1907.09417. arXiv:1907.09417 <http://arxiv.org/abs/1907.09417v1>
- [9] Alessio Conte, Daniele De Sensi, Roberto Grossi, Andrea Marino, and Luca Versari. 2020. Truly Scalable K-Truss and Max-Truss Algorithms for Community Detection in Graphs. *IEEE Access* 8 (2020), 139096–139109. <https://doi.org/10.1109/ACCESS.2020.3011667>
- [10] Yan S. Couto and Cristina G. Fernandes. 2026. Hardness of Dynamic Core and Truss Decompositions. In *Lecture Notes in Computer Science*. Springer Nature Switzerland, 64–80. [https://doi.org/10.1007/978-3-032-06706-7\\_5](https://doi.org/10.1007/978-3-032-06706-7_5)
- [11] Laxman Dhulipala, George Z. Li, and Quanquan C. Liu. 2023. Near-Optimal Differentially Private k-Core Decomposition. arXiv:2312.07706. arXiv:2312.07706 <http://arxiv.org/abs/2312.07706v2>
- [12] Kasimir Gabert, Ali Pinar, and Ümit V. Çatalyürek. 2022. Batch Dynamic Algorithm to Find k-Cores and Hierarchies. arXiv:2203.13095. (2022). arXiv:2203.13095 <http://arxiv.org/abs/2203.13095v1>
- [13] Bin Guo and Emil Sekerinski. 2023. Parallel Order-Based Core Maintenance in Dynamic Graphs. In *Proceedings of the 52nd International Conference on Parallel Processing*. ACM, 122–131. <https://doi.org/10.1145/3605573.3605597>
- [14] Chuhan Hu, Ming Zhong, and Lei Li. 2026. Querying Cohesive Subgraph regarding Span-Constrained Triangles on Temporal Graphs with Dynamic Index Maintenance. In *arXiv (Cornell University)*. Cornell University. <https://arxiv.org/pdf/2606.11582>
- [15] Xin Huang and Laks V. S. Lakshmanan. 2016. Attribute Truss Community Search. arXiv:1609.00090. (2016). arXiv:1609.00090 <http://arxiv.org/abs/1609.00090v3>
- [16] Venkata Rohit Jakkula and George Karypis. 2019. Streaming and Batch Algorithms for Truss Decomposition. In *2019 First International Conference on Graph Computing (GC)*. IEEE, 51–59. <https://doi.org/10.1109/gc46384.2019.00016>
- [17] Humayun Kabir and Kamesh Madduri. 2017. Shared-Memory Graph Truss Decomposition. In *2017 IEEE 24th International Conference on High Performance Computing (HiPC)*. IEEE, 13–22. <https://doi.org/10.1109/hipc.2017.00012>
- [18] Tommaso Lanciano, Atsushi Miyauchi, Adriano Fazzino, and Francesco Bonchi. 2024. A Survey on the Densest Subgraph Problem and its Variants. *Comput. Surveys* 56, 8 (2024), 1–40. <https://doi.org/10.1145/3653298>
- [19] Quanquan C. Liu, Jessica Shi, Shangdi Yu, Laxman Dhulipala, and Julian Shun. 2022. Parallel Batch-Dynamic Algorithms for k-Core Decomposition and Related Graph Problems. In *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures*. ACM, 191–204. <https://doi.org/10.1145/3490148.3538569>

- [20] Quanquan C. Liu, Julian Shun, and Igor Zablotchi. 2024. Parallel k-Core Decomposition with Batched Updates and Asynchronous Reads. arXiv:2401.08015. (2024). arXiv:2401.08015 <http://arxiv.org/abs/2401.08015v1>
- [21] Youzhe Liu, Xiaojun Dong, Yan Gu, and Yihan Sun. 2025. Parallel k-Core Decomposition: Theory and Practice. In *Proceedings of the ACM on Management of Data*, Vol. 3. Association for Computing Machinery, 1–27. <https://doi.org/10.1145/3725332>
- [22] Quintino Francesco Lotito and Alberto Montresor. 2020. Efficient Algorithms to Mine Maximal Span-Trusses From Temporal Graphs. International Workshop on Mining and Learning with Graphs 2020 (MLG '20). arXiv:2009.01928 <http://arxiv.org/abs/2009.01928v2>
- [23] Wensheng Luo, Chenhao Ma, Yixiang Fang, and Laks V. S. Lakshman. 2023. A Survey of Densest Subgraph Discovery on Large Graphs. (2023). <https://doi.org/10.48550/arxiv.2306.07927>
- [24] Wensheng Luo, Qiaoyuan Yang, Yixiang Fang, and Xu Zhou. 2023. Efficient Core Maintenance in Large Bipartite Graphs. In *Proceedings of the ACM on Management of Data*, Vol. 1. Association for Computing Machinery, 1–26. <https://doi.org/10.1145/3617329>
- [25] Ziwei Mo, Qi Luo, Dongxiao Yu, Hao Sheng, Jiguo Yu, and Xiuzhen Cheng. 2023. Distributed Truss Computation in Dynamic Graphs. *Tsinghua Science and Technology* 28, 5 (2023), 873–887. <https://doi.org/10.26599/tst.2022.9010019>
- [26] Ahmet Erdem Saryüce. 2025. A powerful lens for temporal network analysis: temporal motifs. *Discover Data* 3, 1 (2025). <https://doi.org/10.1007/s44248-025-00032-8>
- [27] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2023. Parallel Algorithms for Hierarchical Nucleus Decomposition. (2023). <https://doi.org/10.48550/arxiv.2306.08623>
- [28] Shaden Smith, Xing Liu, Nesreen K. Ahmed, Ancy Sarah Tom, Fabrizio Petrini, and George Karypis. 2017. Truss decomposition on shared-memory parallel systems. In *2017 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–6. <https://doi.org/10.1109/hpec.2017.8091049>
- [29] Zitan Sun, Xin Huang, Qing Liu, and Jianliang Xu. 2023. Efficient Star-based Truss Maintenance on Dynamic Graphs. In *Proceedings of the ACM on Management of Data*, Vol. 1. Association for Computing Machinery, 1–26. <https://doi.org/10.1145/3589278>
- [30] Zitan Sun, Xin Huang, Jianliang Xu, Francesco Bonchi, and Lijun Chang. 2025. Probabilistic Truss Decomposition on Uncertain Graphs: Indexing and Dynamic Maintenance. *ACM Transactions on Database Systems* 50, 2 (2025), 1–40. <https://doi.org/10.1145/3721428>
- [31] Jia Wang and James Cheng. 2012. Truss decomposition in massive networks. *Proceedings of the VLDB Endowment* 5, 9 (2012), 812–823. <https://doi.org/10.14778/2311906.2311909>
- [32] Jian Wu, Alison Goshulak, Venkatesh Srinivasan, and Alex Thomo. 2018. K-Truss Decomposition of Large Networks on a Single Consumer-Grade Machine. In *2018 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*. IEEE, 873–880. <https://doi.org/10.1109/asonam.2018.8508642>
- [33] Yanwei Zheng, Zichun Zhang, Qi Luo, Zhenzhen Xie, and Dongxiao Yu. 2023. A Truss-Based Framework for Graph Similarity Computation. *Journal of Database Management* 34, 1 (2023), 1–18. <https://doi.org/10.4018/jdm.322087>
- [34] Rui Zhou, Chengfei Liu, Jeffrey Xu Yu, Weifa Liang, and Yanchun Zhang. 2014. Efficient Truss Maintenance in Evolving Networks. arXiv:1402.2807. (2014). arXiv:1402.2807